

```
1 #include "maps.h"
2
3
4 int min(int a, int b) {
5     if (a >= b) return b;
6     return a;
7 }
8
9 void random_shuffle_array(int* arr, int n) {
10     for (int i = n - 1; i > 0; i--) {
11         int j = rand() % (i + 1);
12         int temp = arr[i];
13         arr[i] = arr[j];
14         arr[j] = temp;
15     }
16 }
17
18 int* get_random_subset(int m, int n) {
19     int* full = malloc(n * sizeof(int));
20     for (int i = 0; i < n; i++) {
21         full[i] = i;
22     }
23
24     random_shuffle_array(full, n);
25
26     int* subset = malloc(m * sizeof(int));
27     for (int i = 0; i < m; i++) {
28         subset[i] = full[i];
29     }
30
31     free(full);
32     return subset;
33 }
34
35 int* get_random_permutation(int m) {
36     int* perm = malloc(m * sizeof(int));
37     for (int i = 0; i < m; i++) {
38         perm[i] = i;
39     }
40     random_shuffle_array(perm, m);
41     return perm;
42 }
43
44 int** get_binomial(int n) {
45     int N = n+1;
46     int** res = malloc(N*sizeof(int*));
47     for (int i = 0; i < N; i++) {
48         res[i] = calloc(N, sizeof(int));
49     }
50
51     for (int m = 0; m < N; m++) {
52         res[0][m] = 1;
53     }
54
55     for (int k = 1; k < N; k++) {
56         for (int m = k; m < N; m++) {
57             res[k][m] = res[k-1][m-1] + res[k][m-1];
58         }
59     }
60 }
```

```
61     return res;
62 }
63
64 void free_matrix(int** matrix, int n) {
65     assert(matrix != NULL);
66     for (int i = 0; i < n+1; i++) {
67         assert(matrix[i] != NULL);
68         free(matrix[i]);
69     }
70     free(matrix);
71 }
72
73 void print_matrix(int** matrix, int n) {
74     for (int i = 0; i < n; i++) {
75         for (int j = 0; j < n; j++) {
76             printf("%d ", matrix[i][j]);
77         }
78         printf("\n");
79     }
80 }
81
82 long factorial(int n) {
83     int res = 1;
84     for (int i = 2; i <= n; i++) {
85         res *= i;
86     }
87     return res;
88 }
89
90 long long n_choose_k(int n, int k) {
91     if (k < 0 || k > n) return 0;
92     if (k > n - k) k = n - k;
93
94     long long result = 1;
95     for (int i = 1; i <= k; i++) {
96         result = result * (n - k + i) / i;
97     }
98     return result;
99 }
100
101 void print_map(int* map, int n) {
102     for (int i = 0; i < n; i++) {
103         printf("%d -> %d\n", i, map[i]);
104     }
105     printf("\n");
106 }
107
108 void print_arr(int* arr, int n) {
109     for (int i = 0; i < n; i++) {
110         printf("%d ", arr[i]);
111     }
112     printf("\n");
113 }
114
115 int* get_inverse_perm(int* p, int n) {
116     int* inv = malloc(n*sizeof(int));
117     for (int i = 0; i < n; i++) {
118         inv[p[i]] = i;
119     }
120     return inv;
```

```
121 }
122
123 bool* init_subset(int m, int n) {
124     bool* res = calloc(n, sizeof(bool));
125     for (int i = 0; i < m; i++) {
126         res[i] = true;
127     }
128     return res;
129 }
130
131 bool is_last_subset(bool* subset, int m, int n) {
132     for (int i = n-1; i >= n-m; i--) {
133         if (!subset[i]) {
134             return false;
135         }
136     }
137     return true;
138 }
139
140 void push_to_subset(list_t* sub, int i, int v, int k, int n, int** binom) {
141     if (k <= 0 || n <= 0 || k > n) return;
142     else {
143         int choice = binom[k-1][n-1];
144         if (i < choice) {
145             push(sub, v);
146             push_to_subset(sub, i, v+1, k-1, n-1, binom);
147         } else {
148             push_to_subset(sub, i-choice, v+1, k, n-1, binom);
149         }
150     }
151 }
152
153 list_t* get_subset_list_from_binom(int i, int m, int n, int** binom) {
154     assert(binom != NULL);
155     list_t* sub = create_empty_list();
156
157     push_to_subset(sub, i, 0, m, n, binom);
158
159     return sub;
160 }
161
162 list_t* get_subset_list(int i, int m, int n) {
163     int** binom = get_binomial(n);
164     list_t* sub = create_empty_list();
165
166     push_to_subset(sub, i, 0, m, n, binom);
167     free_matrix(binom, n);
168
169     return sub;
170 }
171
172 int* array_from_list(list_t* l) {
173     assert(l != NULL);
174     int n = l->length;
175     int* res = malloc(n*sizeof(int));
176     node_t* c = l->head;
177     for (int i = 0; i < n; i++) {
178         res[i] = c->val;
179         c = c->next;
180     }
181 }
```

```
181
182     return res;
183 }
184
185 int* map_from_list(list_t* image) {
186     assert(image != NULL);
187     int n = image->length;
188     int* res = malloc(n*sizeof(int));
189     node_t* c = image->head;
190     for (int i = 0; i < n; i++) {
191         res[i] = c->val;
192         c = c->next;
193     }
194
195     return res;
196 }
197
198 int* get_array_from_subset_binom(int i, int m, int n, int** binom) {
199     list_t* subset = get_subset_list_from_binom(i, m, n, binom);
200     int *array = array_from_list(subset);
201     free_list(subset);
202     return array;
203 }
204
205 void get_next_subset(bool* set, int m, int n) {
206     int i = n - 1;
207     while (i >= 0 && set[i]) {
208         i--;
209     }
210     if (i >= 0) {
211         int j = i - 1;
212         while (j >= 0 && !set[j]) {
213             j--;
214         }
215
216         if (j >= 0) {
217             set[j] = false;
218             set[j + 1] = true;
219
220             int remaining = 0;
221             for (int k = j + 2; k < n; k++) {
222                 if (set[k]) {
223                     remaining++;
224                     set[k] = false;
225                 }
226             }
227
228             for (int k = 0; k < remaining; k++) {
229                 set[j + 2 + k] = true;
230             }
231         }
232     }
233 }
234
235 int* subset_to_map(bool* set, int m, int n) {
236     int* map = calloc(m, sizeof(int));
237     int index = 0;
238     for (int x = 0; x < n; x++) {
239         if (set[x]) {
240             map[index] = x;
```

```
241     index++;
242 }
243 }
244 return map;
245 }
246
247 bool* map_to_subset(int* map, int m, int n) {
248     bool* res = calloc(n, sizeof(bool));
249     int to_add;
250     for (int i = 0; i < m; i++) {
251         to_add = map[i];
252         res[to_add] = true;
253     }
254
255     return res;
256 }
257
258 int* free_get_next_map(int* map, int m, int n) {
259     bool* curr_subset = map_to_subset(map, m, n);
260     get_next_subset(curr_subset, m, n);
261     free(map);
262     return subset_to_map(curr_subset, m, n);
263 }
264
265 int* get_next_map(int* map, int m, int n) {
266     bool* curr_subset = map_to_subset(map, m, n);
267     get_next_subset(curr_subset, m, n);
268     return subset_to_map(curr_subset, m, n);
269 }
270
271 int calculate_from_list(group_t* g, list_t* l) {
272     assert(g != NULL && l != NULL);
273     int res = -1;
274     for (node_t* c = l->head; c != NULL; c = c->next) {
275         res = calculate(g, res, c->val);
276     }
277
278     return res;
279 }
280
281 int get_extension_value(group_t* g2, list_t* l, int* partial_map) {
282     assert(g2 != NULL && l != NULL && partial_map != NULL);
283     int res = -1;
284     int y;
285     for (node_t* c = l->head; c != NULL; c = c->next) {
286         y = partial_map[c->val];
287         res = calculate(g2, res, y);
288     }
289     return res;
290 }
291
292 void map_image_generators(int* map, list_t* gen, int* image) {
293     int index = 0;
294     int curr_gen;
295     int curr_image;
296     for (node_t* c = gen->head; c != NULL; c=c->next) {
297         curr_gen = c->val;
298         curr_image = image[index];
299         map[curr_gen] = curr_image;
300         index++;
    }
```

```
301     }
302 }
303
304 void map_elements(int* map, list_t* l1, list_t* l2) {
305     assert(map != NULL && l1 != NULL && l2 != NULL);
306     assert(l1->length == l2->length);
307     int n = l1->length;
308     node_t* c1 = l1->head;
309     node_t* c2 = l2->head;
310     for (int i = 0; i < n; i++) {
311         map[c1->val] = c2->val;
312         c1 = c1->next;
313         c2 = c2->next;
314     }
315 }
316
317 int* get_full_map(gdata_t* g1_data, group_t* g2, int* image_gen, int m) {
318     assert(g1_data != NULL && g2 != NULL && image_gen != NULL);
319     int n = g2->n;
320     int* map = calloc(n, sizeof(int));
321     for (int i = 0; i < n; i++) {
322         map[i] = -1;
323     }
324
325     map_image_generators(map, g1_data->gen, image_gen);
326
327     int y;
328     list_t* decomp;
329     for (int x = 0; x < n; x++) {
330         if (map[x] == -1) {
331             decomp = g1_data->rel->decomps[x];
332             y = get_extension_value(g2, decomp, map);
333             assert(y >= 0 && y < n);
334             map[x] = y;
335         }
336     }
337
338     return map;
339 }
340
341 int* get_induced_map(gdata_t* g1_data, group_t* g2, list_t* image_gen, int m) {
342     assert(g1_data != NULL && g2 != NULL && image_gen != NULL);
343     assert(image_gen->length == g1_data->gen->length && g1_data->gen->length == m);
344     int n = g2->n;
345     int* map = calloc(n, sizeof(int));
346     for (int i = 0; i < n; i++) {
347         map[i] = -1;
348     }
349
350     map_elements(map, g1_data->gen, image_gen);
351
352     int y;
353     list_t* decomp;
354     for (int x = 0; x < n; x++) {
355         if (map[x] == -1) {
356             decomp = g1_data->rel->decomps[x];
357             y = get_extension_value(g2, decomp, map);
358             assert(y >= 0 && y < n);
359             map[x] = y;
360         }
361     }
```

```
361     }
362
363     return map;
364 }
365
366 void rec_generate_permutation_into(int n, llong k, int *res) {
367     if (n == 0) return;
368     int first_elt = k%n;
369     llong inner_k = k/n;
370     res[0] = first_elt;
371     rec_generate_permutation_into(n-1, inner_k, res + 1);
372     for (int i = 1; i < n; i++) {
373         if (res[i] >= first_elt) {
374             res[i]++;
375         }
376     }
377 }
378
379 int* get_kth_permutation_rec(int n, llong k) {
380     int* res = malloc(n*sizeof(int));
381     rec_generate_permutation_into(n, k, res);
382     return res;
383 }
384
385 void apply_permutation(int* perm, int* array, int n) {
386     int* copy = malloc(n*sizeof(int));
387     for (int j = 0; j < n; j++) {
388         copy[j] = array[j];
389     }
390     int new_index;
391     for (int i = 0; i < n; i++) {
392         new_index = perm[i];
393         array[i] = copy[new_index];
394     }
395     free(copy);
396 }
397
398 int* get_from_permutation(int* perm, int* array, int n) {
399     int* permuted = malloc(n*sizeof(int));
400     int new_index;
401     for (int i = 0; i < n; i++) {
402         new_index = perm[i];
403         permuted[i] = array[new_index];
404     }
405     return permuted;
406 }
407
408 int* get_induced_hom(group_t* g2, gdata_t* data, int* image_gen, int m) {
409     assert(m == data->gen->length);
410     int n = g2->n;
411     int* map = malloc(n*sizeof(int));
412     for (int i = 0; i < n; i++) {
413         map[i] = -1;
414     }
415
416     list_t* gen = data->gen;
417     gentabl_t* rel = data->rel;
418
419     int index = 0;
420     for (node_t* c = gen->head; c != NULL; c = c->next) {
```

```
421     map[c->val] = image_gen[index];
422     index++;
423 }
424
425 int y;
426 list_t* decomp;
427 for (int x = 0; x < n; x++) {
428     if (map[x] == -1) {
429         decomp = rel->decomps[x];
430         y = get_extension_value(g2, decomp, map);
431
432         map[x] = y;
433     }
434 }
435
436 return map;
437 }
438
439 bool is_isomorphism(group_t* g1, gdata_t* gdata, group_t* g2, int* map, int n) {
440     assert(g1 != NULL && gdata != NULL && g2 != NULL && map != NULL);
441     assert(g1->n == g2->n && g1->n == n);
442     int y1, y2;
443
444     bool* seen = calloc(n, sizeof(bool));
445     int index;
446     for (int x = 0; x < n; x++) {
447         index = map[x];
448         assert(index >= 0 && index < n);
449         seen[index] = true;
450     }
451     for (int x = 0; x < n; x++) {
452         if (!seen[x]) {
453             free(seen);
454             return false;
455         }
456     }
457     free(seen);
458
459     for (int i = 0; i < min(10, n); i++) {
460         int x = rand() % n;
461         int y = rand() % n;
462         if (map[g1->table[x][y]] != g2->table[map[x]][map[y]]) {
463             return false;
464         }
465     }
466
467     int l, r;
468     for (int x = 0; x < n; x++) {
469         for (int y = 0; y < n; y++) {
470             l = map[g1->table[x][y]];
471             r = g2->table[map[x]][map[y]];
472             if (l != r) {
473                 return false;
474             }
475         }
476     }
477     return true;
478 }
479
```



```
480 bool is_isomorphism_with_order(group_t* g1, gdata_t* gdata, group_t* g2, int*
orders_g1, int* orders_g2, int* map, int n) {
481     assert(g1 != NULL && gdata != NULL && g2 != NULL && map != NULL);
482     assert(orders_g1 != NULL && orders_g2 != NULL);
483     assert(g1->n == g2->n && g1->n == n);
484
485     for (int x = 0; x < n; x++) {
486         if (orders_g1[x] != orders_g2[map[x]]) {
487             return false;
488         }
489     }
490     bool* seen = calloc(n, sizeof(bool));
491     for (int x = 0; x < n; x++) {
492         seen[map[x]] = true;
493     }
494     for (int x = 0; x < n; x++) {
495         if (!seen[x]) {
496             free(seen);
497             return false;
498         }
499     }
500     free(seen);
501     int l, r;
502     for (int x = 0; x < n; x++) {
503         for (int y = 0; y < n; y++) {
504             l = map[g1->table[x][y]];
505             r = g2->table[map[x]][map[y]];
506             if (l != r) {
507                 return false;
508             }
509         }
510     }
511     return true;
512 }
513
514
515
516 bool next_injective_map(int *map, bool *used, int m, int n) {
517     int i = m - 1;
518
519     while (i >= 0) {
520         if (map[i] != -1) {
521             used[map[i]] = false;
522         }
523
524         bool found = false;
525         for (int x = map[i] + 1; x < n; x++) {
526             if (!used[x]) {
527                 map[i] = x;
528                 used[x] = true;
529                 found = true;
530                 break;
531             }
532         }
533
534         if (found) {
535             for (int j = i + 1; j < m; j++) {
536                 map[j] = -1;
537                 for (int x = 0; x < n; x++) {
538                     if (!used[x]) {
```

```
539         map[j] = x;
540         used[x] = true;
541         break;
542     }
543 }
544 }
545     return true;
546 }
547
548     map[i] = -1;
549     i--;
550 }
551 for (int i = 0; i < m; i++) {
552     assert(map[i] >= 0 && map[i] < n);
553 }
554
555     return false;
556 }
557
```